

Examen Práctico: "DataDesk Helpdesk System"

Desarrollo de una Aplicación de Escritorio Profesional con Python y Tkinter

El Reto

Las empresas del sector tecnológico necesitan herramientas ágiles para gestionar las incidencias internas de sus clientes y empleados. Un script por consola se queda corto para las necesidades del mercado actual: **necesitamos interfaces gráficas intuitivas, código mantenible y almacenamiento persistente**.

Tu objetivo en este proyecto es construir **DataDesk**, una aplicación de escritorio completa para la gestión de tickets de soporte técnico (Helpdesk). Deberás diseñar tanto la interfaz de usuario como la arquitectura interna del sistema siguiendo el patrón de diseño modular **Separación de Responsabilidades (SoC)**.

Este proyecto formará parte de tu **portafolio profesional** y estará listo para publicarse en tu perfil de **GitHub**, demostrando tus habilidades en **Python avanzado, POO, Interfaces Gráficas (GUI) y persistencia de datos**.

Requisitos Técnicos del Software

El proyecto se dividirá estrictamente en **tres módulos clave**:

Plaintext

helpdesk_project/

— models.py	# Capa de Lógica de Negocio y Persistencia (POO)
— views.py	# Capa de Interfaz Gráfica (Tkinter / TTK)
— main.py	# Punto de Entrada de la Aplicación

1. Capa de Datos y Lógica (**models.py**)

- **Clase Ticket**: Representa la entidad del problema (ID, Usuario, Descripción, Categoría, Prioridad, Estado). Incluye métodos para convertir el objeto a diccionario y viceversa.
- **Clase TicketManager**: Encargada de la lógica del CRUD (Crear, Leer, Actualizar estado y Eliminar) y la gestión del archivo **tickets.json** para garantizar la **persistencia de datos**.

2. Capa de Interfaz Gráfica (**views.py**)

- **Formulario de Entrada:** Con campos validados (**Entry**, **Combobox**) para dar de alta nuevas incidencias.
- **Visualización Interactiva:** Uso del componente **ttk.Treeview** (tabla) para listar todos los registros.
- **Filtro en Tiempo Real:** Un buscador que filtre las incidencias dinámicamente según la entrada del usuario.
- **Panel Métrico:** Cálculo automático del total de tickets, pendientes y resueltos.
- **Manejo de Alertas:** Mensajes flotantes (**messagebox**) para validaciones y confirmaciones de borrado.

3. Punto de Entrada (**main.py**)

- Módulo principal que orquesta la carga del gestor de datos, la ventana raíz de Tkinter y la ejecución del programa.

Entregables para el Portafolio (GitHub)

Para que tu proyecto cause un gran impacto visual en tu CV y perfil profesional, deberás entregar un **repositorio de GitHub** que contenga:

1. **Código Fuente:** Estructurado, limpio, comentado y con tipado estático (*Type Hints*).
2. **Archivo **README.md** Profesional:**
 - Título descriptivo y resumen del proyecto.
 - **Captura de pantalla o GIF** de la aplicación en funcionamiento.
 - Lista de tecnologías utilizadas (Python 3.x, Tkinter, JSON).
 - Guía paso a paso para clonar y ejecutar el repositorio.

Criterios de Evaluación

- **[30%] Arquitectura y POO:** Estructura modular correcta (**models.py**, **views.py**, **main.py**) y uso de clases.
- **[30%] Funcionalidad y Persistencia:** El CRUD completo funciona sin errores y guarda los cambios en el fichero JSON.
- **[20%] Diseños e Interfaz (GUI):** Usabilidad, disposición ordenada con frames y filtrado dinámico funcional.
- **[20%] Presentación en GitHub:** Calidad del repositorio, código limpio y archivo **README.md** atractivo.